

LangSmith vs Langfuse

Same agent. Same four test questions. Same `gpt-4o` judge. The only thing that changes is which tool is watching.

Every screenshot below is from this lab. Every number is measured, not quoted from a docs page.

- **LangSmith**: hosted, at `smith.langchain.com`
- **Langfuse**: self-hosted, six Docker containers on `localhost:3000`, `v4.6.0 OSS`

What we are watching

A small LangGraph agent with three tools:

- `calculator` , always works
- `get_weather` , fake data that **fails 20% of the time on purpose**
- `search_docs` , tiny RAG over five markdown files

The 20% failure is the interesting part. It is what makes the error screens below real instead of staged.

Contents

Document	Covers
01-tracing.md	Turning tracing on, reading a trace, error handling, grouping runs
02-evaluation.md	Golden datasets, LLM-as-judge, measured scores
03-features.md	Dashboards, and what each tool has that the other does not
04-cost.md	Pricing for both, and measured CPU/RAM/disk for self-hosted Langfuse

Running this yourself

```
# plain app, traced by LangSmith automatically
uv run python -m src.agent "what's the weather in Chennai and what's 100/7?"

# same agent, explicitly traced into Langfuse
uv run python -m src.obs_langfuse.traced_run "what's the weather in Chennai and what's 100/7?"

# datasets (run once each)
uv run python -m src.obs_langsmith.dataset
uv run python -m src.obs_langfuse.dataset

# evaluations
uv run python -m src.obs_langsmith.evaluate
uv run python -m src.obs_langfuse.evaluate
```

To force a failure instead of waiting for the 20% dice:

```
PYTHONPATH=. uv run python -c "
import random
random.random = lambda: 0.0
from dotenv import load_dotenv; load_dotenv()
from src.agent.graph import graph
graph.invoke({'messages': [('user', 'what is the weather in Chennai?')]}, config={'run_name': 'for
"
```

Verdict

What you care about	Pick
Tracing working in ten seconds	LangSmith
Data must not leave your network	Langfuse
Seeing <i>where</i> an agent broke	Langfuse
Judge explaining its scores	Langfuse
Nothing to operate	LangSmith
Filtering on your own metadata keys	LangSmith

There is no overall winner. If your data can leave the building, LangSmith gets you further in an afternoon and there is nothing to maintain. If it cannot, Langfuse is a real replacement rather than a downgrade, and its error views are genuinely better. The price is six containers and a port conflict.

Tracing and debugging

How each tool is switched on, what a run looks like, and what happens when the agent breaks.

Turning tracing on

This is the biggest difference, and it is visible in the code.

LangSmith: one environment variable

```
LANGSMITH_TRACING=true
```

That is it. No imports, no code changes, nothing passed into the agent.

Proof is in the run list. The rows named `LangGraph` came from running the plain app:

```
uv run python -m src.agent "what's the weather in Chennai and what's 100/7?"
```

That file does not mention LangSmith anywhere. It still shows up with full latency, token and cost data.

Personal / Tracing / ilm-observability-lab

ilm-observability-lab ID Tracing Evaluators Automations Insights Engine New

Default View Last 1 day Threads Traces Runs Input Search input content

Add filter

2

nts 1

Beta

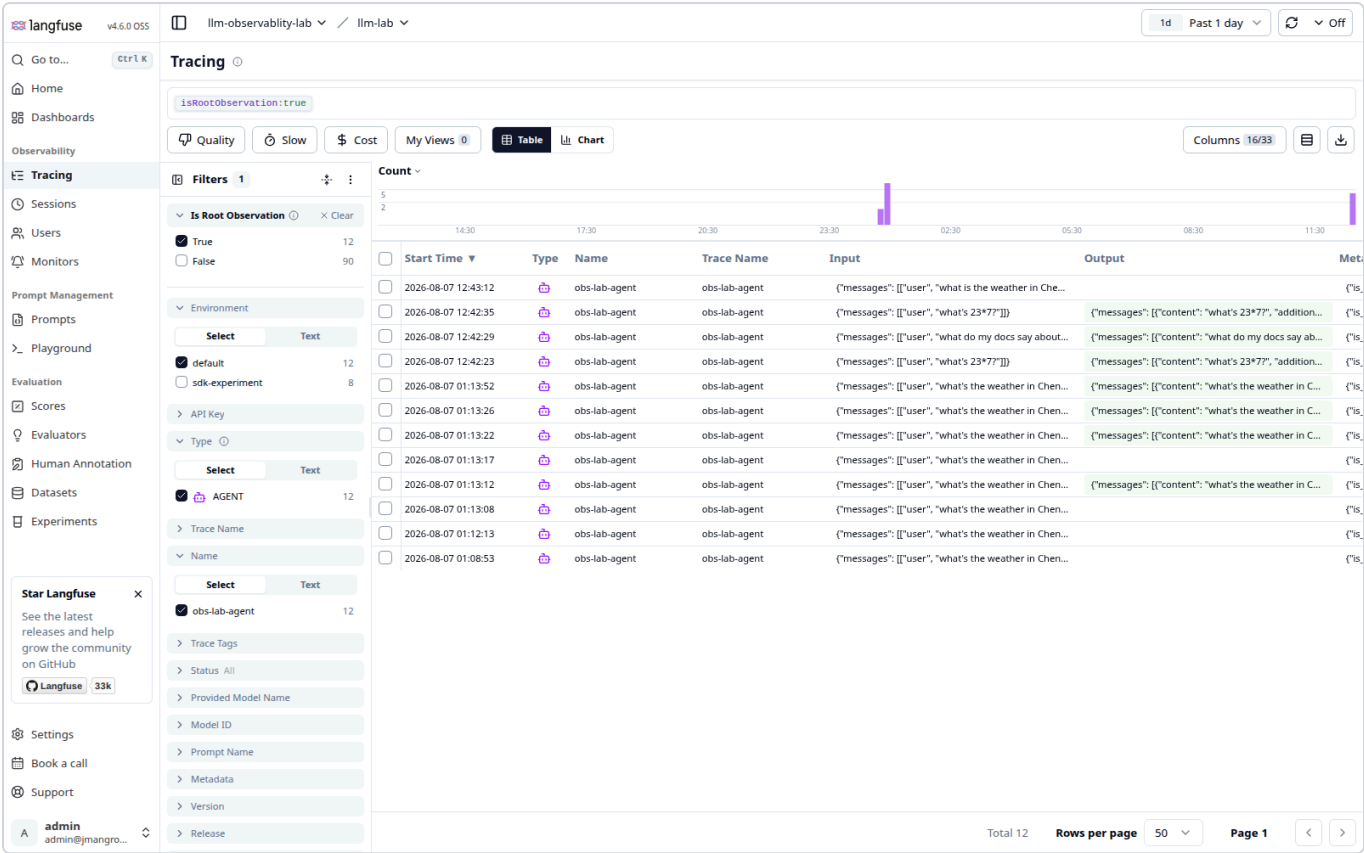
☐	☒	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation ...	Tokens	Cost	First Token	Tags
☐	☒	forced-weather-failure	user: what is the weather ...		RuntimeError("weather s...	8/7/2026, 11:51...	1.36s			279	\$0.0008175		
☐	☒	LangGraph	user: what's the weather ...	ai: The weather in Chenn...		8/7/2026, 11:4...	2.13s			690	\$0.002295		
☐	☒	obs-lab-tagged-run	user: what's the weather ...	ai: The weather in Chenn...		8/7/2026, 11:41...	2.77s			690	\$0.002295		phase3
☐	☒	LangGraph	user: what do my docs s...	ai: Your docs say to retry ...		8/7/2026, 11:35...	3.96s			878	\$0.002795		
☐	☒	obs-lab-tagged-run	user: what's the weather ...	ai: The weather in Chenn...		8/7/2026, 11:3...	2.18s			690	\$0.002295		manual-te
☐	☒	ChatOpenAI	human: You are grading ...	ai: CORRECT The agent's...		8/7/2026, 11:17...	0.54s			130	\$0.0004675		
☐	☒	LangGraph	user: what's 23*7?	ai: 23 * 7 equals 161.		8/7/2026, 11:17...	0.99s			577	\$0.001645		
☐	☒	LangGraph	user: what's the weather ...		RuntimeError("weather s...	8/7/2026, 11:17...	0.63s			317	\$0.0011525		
☐	☒	ChatOpenAI	human: You are grading ...	ai: CORRECT The agent's...		8/7/2026, 11:17...	0.62s			236	\$0.0008225		
☐	☒	LangGraph	user: what do my docs s...	ai: Your docs say to retry ...		8/7/2026, 11:17...	3.26s			878	\$0.002795		
☐	☒	LangGraph	user: weather in Mumbai,...		RuntimeError("weather s...	8/7/2026, 11:17...	1.41s			282	\$0.0008175		
☐	☒	LangGraph	user: what's the weather ...		RuntimeError("weather s...	8/7/2026, 11:15...	0.66s			317	\$0.0011525		
☐	☒	ChatOpenAI	human: You are grading ...	ai: CORRECT The agent's...		8/7/2026, 11:15...	0.56s			133	\$0.000475		
☐	☒	LangGraph	user: what's 23*7?	ai: The result of 23 * 7 is 1...		8/7/2026, 11:15...	1.11s			581	\$0.001685		
☐	☒	ChatOpenAI	human: You are grading ...	ai: First line: CORRECT Se...		8/7/2026, 11:15...	0.78s			288	\$0.0009975		
☐	☒	LangGraph	user: what do my docs s...	ai: Your docs say: - Retry ...		8/7/2026, 11:15...	14.07s			924	\$0.003255		

Look at the columns you get for free: `Latency` , `Tokens` , `Cost` , `Error` , `Tags` . The red rows are the weather tool failing. Nobody wrote error handling to make those appear.

Langfuse: one handler per call

```
handler = CallbackHandler()
graph.invoke(..., config={"callbacks": [handler]})
```

Nothing is recorded unless you pass that handler.



Which is better depends on what you care about. LangSmith gets you tracing in ten seconds, but it also captures everything in the process, including code you did not write. Langfuse costs you a line at every call site, and in exchange nothing leaves the process unless you asked for it.

One useful side effect: with both sets of keys in `.env`, a single run lands in **both** tools at once. Worth doing once.

What a single run looks like

Both give you the same core thing: a tree of what happened, with time and tokens on each node.

LangSmith

Personal / Tracing / ilm-observability-lab

LangGraph ID

Tracing

LangGraph 11:44 AM

2.13s 690 / \$0.0023

agent

1.55s 317 graph:step:1

ChatOpenAI gpt-4o

1.26s 317 seq:step:1

should_continue

0.00s seq:step:3

tools

0.00s graph:step:2

get_weather

0.00s seq:step:1

calculator

0.00s seq:step:1

agent

0.57s 373 graph:step:3

ChatOpenAI gpt-4o

0.57s 373 seq:step:1

should_continue

0.00s seq:step:3

Attributes

Metadata

revision_id dc6b26a-dirty

LangSmith

LANGSMITH_ENDPOINT <https://api.smith.langchain.com>

LANGSMITH_PROJECT ilm-observability-lab

LANGSMITH_TRACING true

ls_integration langgraph

ls_run_depth 0

Runtime

langchain_core_version 15.3

langchain_version 1.3.14

library langchain-core

library_version 15.3

platform Linux-6.17.0-1030-oem-x86_64-with-glibc2.39

py_implementation CPython

runtime python

runtime_version 3.12.3

sdk langsmith-py

sdk_version 0.10.16

2.13s , 690 tokens, \$0.0023 at the root. The graph:step and seq:step labels come from LangGraph and show the execution order.

Langfuse

langfuse v4.6.0 OSS

Go to...

Home

Dashboards

Observability

Tracing

Sessions

Users

Monitors

Prompt Management

Prompts

Playground

Evaluation

Scores

Evaluators

Human Annotation

Datasets

Experiments

Star Langfuse

See the latest releases and help grow the community on GitHub

Langfuse 33k

Settings

Book a call

Support

admin admin@mangro...

ilm-observability-lab / ilm-lab / Traces

Trace obs-lab-agent: 696c2cfd5732276e3ecb0cb08aecedab3

2026-08-07 12:42:35.076

Latency: 1.87s Session: session-001 User ID: vikash Env: default \$0.001645 550 prompt - 27 completion (\$ 577)

Preview Log View Scores Formatted JSON

Tags manual-test phase4

Input

Path	Value
json	2 items
0	"user"
1	"what's 23*7?"

Show 2 more...

Calculator

161

Assistant

23 * 7 equals 161.

Corrected Output

Click to add corrected output

Metadata

Path	Value
is_langchain_root	true
ls_integration	"Langgraph"
scope.attributes.public_key	"pk-1f-492508f9-5e47-4c07-9ed4-2c7ba18d7126"
scope.version	"4.14.3"
scope.name	"Langfuse-sdk"
resourceAttributes.service.name	"unknown_service"
resourceAttributes.service.instance.id	"945f5e5f-ac70-4ac9-9704-b0b6c97bab12"
resourceAttributes.telemetry.sdk.version	"1.44.0"
resourceAttributes.telemetry.sdk.name	"opentelemetry"
resourceAttributes.telemetry.sdk.language	"python"

Graph

Aggregated Expanded

start

agent (2/2)

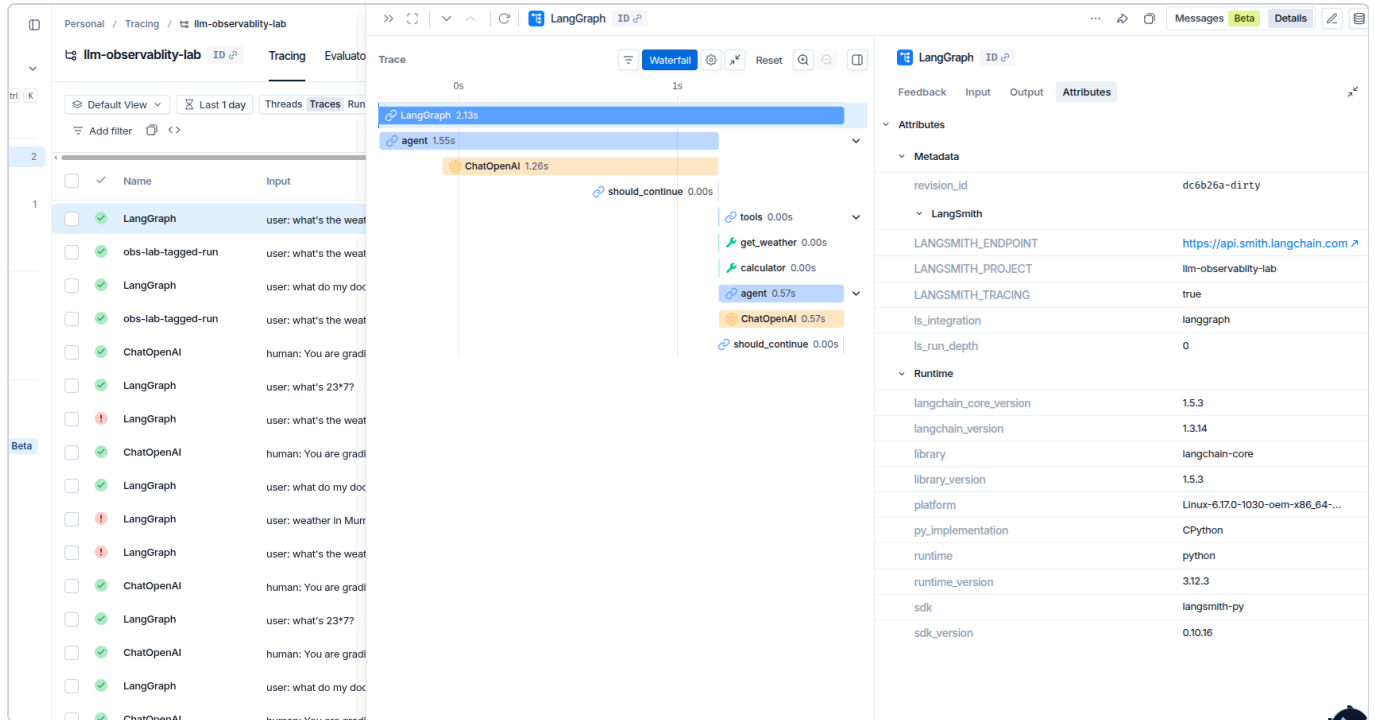
tools

end

Same shape, plus a rendered graph of the agent loop underneath the tree.

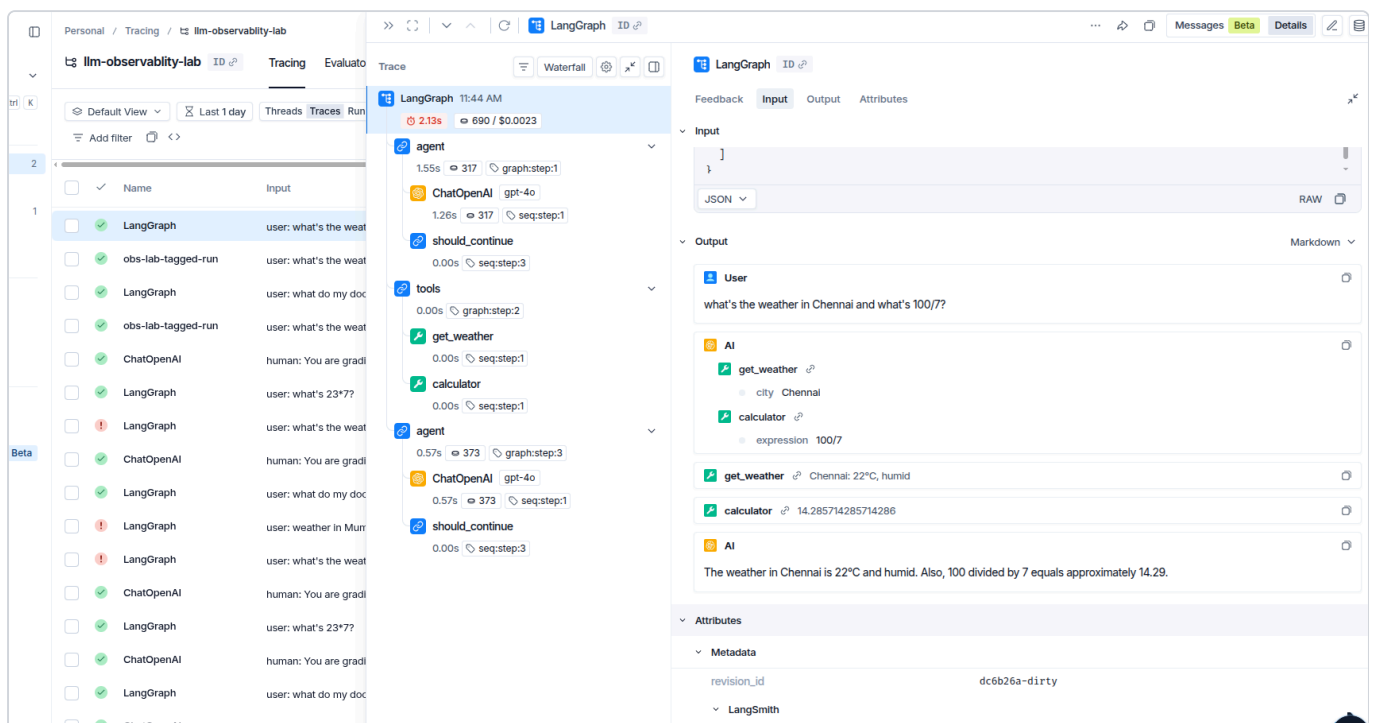
Timing view

LangSmith has a waterfall that makes it obvious where the time actually went:



The whole run is 2.13s and the ChatOpenAI call is 1.26s of it. Both tool calls are 0.00s. The model is the slow part, not your code. That is nearly always the answer, and it is worth seeing once.

The conversation itself

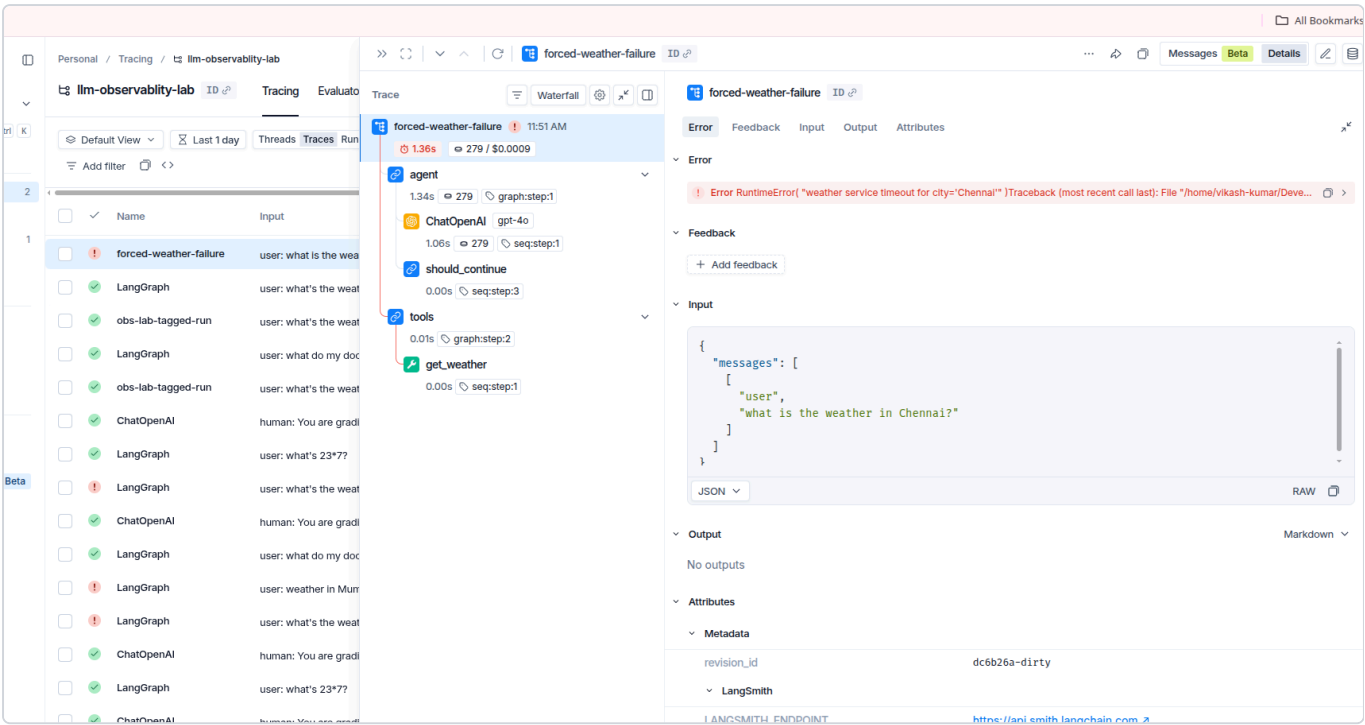


Both tools show the full message flow: what the model asked for, what each tool returned, what came back at the end.

When something breaks

This is where the two tools genuinely differ, and it matters more than any feature list.

LangSmith marks the root of the trace and puts the full Python traceback in a dedicated **Error** tab. Output says **No outputs**.



Langfuse marks every span the error passed through, all the way up.

The screenshot displays the Langfuse interface for a trace titled 'obs-lab-agent: 39d207c301916a86e0eb819c520508e2'. The left sidebar shows the navigation menu with options like Home, Dashboards, Observability, Tracing, Sessions, Users, Monitors, Prompt Management, Prompts, Playground, Evaluation, Scores, Evaluators, Human Annotation, Datasets, and Experiments. The main area shows the trace details, including a tree view of the execution steps. The 'tools' step is highlighted with a red 'ERROR' label. The right panel provides a detailed view of the input and assistant's response, along with metadata.

Read the Langfuse tree carefully:

```
obs-lab-agent  ERROR
  agent
    ChatOpenAI      1.30s  270 -> 48  $0.001155
    should_continue
    tools            ERROR
      get_weather    ERROR  0.00s
      calculator      0.00s
```

`get_weather` failed, so `tools` is marked, so the whole trace is marked. `calculator` sits right next to it, untouched and green. The graph below even paints `__end__` red.

Verdict on errors: Langfuse wins. LangSmith tells you *that* the run failed and hands you a traceback. Langfuse shows you *where* in the agent it failed without you reading a single line of Python.

One detail worth knowing: `get_weather` shows `0.00s`. A real network timeout would take seconds. Zero means it failed on the first line of the function. That is how you tell a fake failure from a real one.

Grouping runs together

Both can thread separate runs into one conversation.

LangSmith calls them Threads:

The screenshot displays the LangSmith interface for a thread titled 'llm-observability-lab'. The left sidebar shows a list of experiments and queues. The main area is divided into 'INPUTS' and 'OUTPUTS' sections. The 'INPUTS' section shows a user message: 'what's the weather in Chennai and what's 100/7?'. The 'OUTPUTS' section shows the AI's response, which includes a 'get_weather' tool call with parameters 'city: Chennai' and a 'calculator' tool call with parameters 'expression: 100/7'. The right sidebar shows the 'Stats' section, including 'Time' (First start: 08/07/2026, 12:40:22 AM GMT+5; Last end: 08/07/2026, 11:41:07 AM GMT+5), 'Latency' (P50: 2.0s, P99: 2.0s), and 'Total cost breakdown' (Input: 74% / 2.3K / 0.4; Output: 26% / 207 / 0.4; Total: 100% / 2.5K / 0.4).

Langfuse calls them Sessions:

The screenshot displays the Langfuse interface for a session titled 'llm-observability-lab'. The left sidebar shows a list of sessions and queues. The main area is divided into 'SESSIONS' and 'TRACES' sections. The 'SESSIONS' section shows a table with columns: ID, Created At, Duration, Environment, Scores, User IDs, and Traces. The table contains one row: session-001, Created At: 2026-08-07 01:08:53, Duration: 11h 34m 20s, Environment: default, User IDs: vikash, Traces: 12. The right sidebar shows the 'Filters' section, including 'Environment' (default, sdk-experiment), 'Session ID', 'User IDs', 'Trace Tags', 'Metadata', 'Bookmarked' (Bookmarked, Not bookmarked), 'Session Duration', 'Traces Count', 'Input Tokens', 'Output Tokens', 'Total Tokens', 'Input Cost', 'Output Cost', 'Total Cost', 'Categorical Scores', 'Numeric Scores', 'Boolean Scores', 'Comment Count', and 'Comment Content'.

One session, session-001 , holding 12 traces over 11 hours, attributed to user vikash .

That 11 hour duration is a warning, not a feature. The session ID is hardcoded in [traced_run.py](#), so every run ever executed piles into the same session. In a real app you generate one per conversation.

The two tools name these fields differently:

	LangSmith	Langfuse
Tags	<code>config["tags"]</code>	<code>metadata["langfuse_tags"]</code>
User	<code>metadata["user_id"]</code> , any key you like	<code>metadata["langfuse_user_id"]</code> , reserved name
Session	<code>metadata["session_id"]</code> , any key you like	<code>metadata["langfuse_session_id"]</code> , reserved name

LangSmith lets you invent your own keys and filter on them. Langfuse only promotes specific `langfuse_` - prefixed keys into real UI features, and treats everything else as plain metadata. You trade magic strings for purpose-built screens.

Datasets and evaluation

Running the same four golden questions through both tools with an LLM judge.

Datasets and evaluation

Both hold a golden dataset and run your agent against it with an LLM judge.

The exact same four questions were loaded into both, imported from one Python list, so the comparison is fair.

langfuse

v4.6.0 OSS

llm-observability-lab / llm-lab / Datasets / llm-obs-lab-golden

+ New item

Upload CSV

Go to...

ctrl K

Home

Dashboards

Observability

Tracing

Sessions

Users

Monitors

Prompt Management

Prompts

Playground

Evaluation

Scores

Evaluators

Human Annotation

Datasets

Experiments

Star Langfuse

See the latest releases and help grow the community on GitHub

Langfuse

33k

Settings

Book a call

Support

admin

admin@jmgro...

Dataset llm-obs-lab-golden

ItemsExperiments

Search...

IDs

Filters

Columns 8/8

Item id	Source	Status	Created At	Input	Expected Output	Metadata	Actions
b6d92353-9d59-4...		Active	2026-08-07 11:15:28	1 Items question: "weather in Mumbai, then multiply the temperature by 2"	1 Items answer: "States Mumbai's current temperature from the weather tool and gives that temperature multiplied by 2."		
dc09870c-1e5e-4...		Active	2026-08-07 11:15:28	1 Items question: "what do my docs say about retries?"	1 Items answer: "Retry only idempotent operations, use exponential backoff with jitter, cap attempts at 3-5, retry on 429/5xx only"		
a0e8a271-4021-4...		Active	2026-08-07 11:15:28	1 Items question: "what's the weather in Chennai and what's 100/7?"	1 Items answer: "Chennai is 22°C and humid; 100/7 is approximately 14.29."		
8c64aa63-eec8-4...		Active	2026-08-07 11:15:28	1 Items question: "what's 23*7?"	1 Items answer: "161"		

Rows per page

50

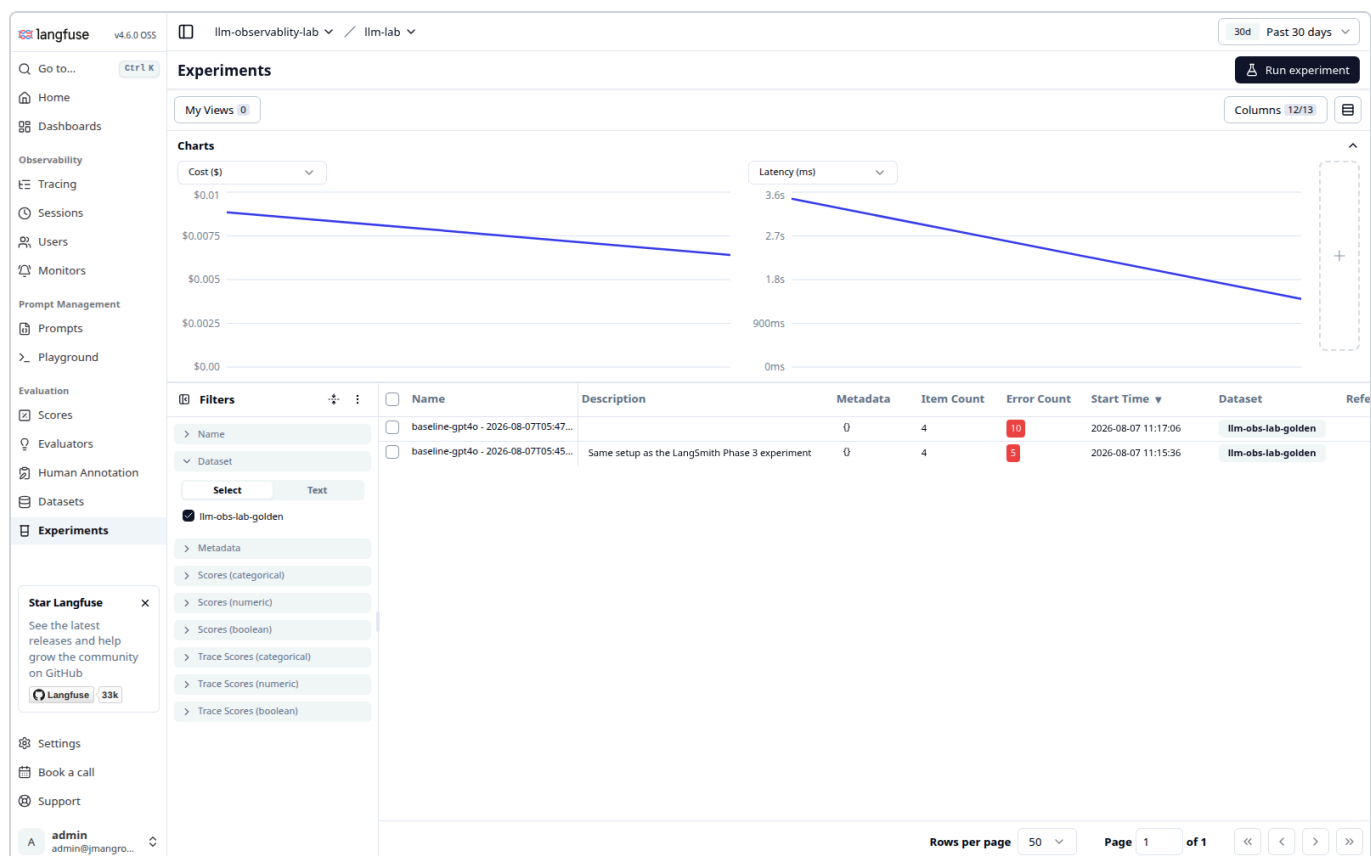
Page 1 of 1

langfuse v4.6.0 OSS llm-observability-lab llm-lab Experiments			
Go to... Experiment Baseline baseline-gpt4o - 2026-08-07T05... Experiment selection Search experiments... Display			
Results Analytics			
My Views Columns 9/12			
Filters Item Metadata Metadata Scores (categorical) Scores (numeric) Scores (boolean) Trace Scores (categorical) Trace Scores (numeric) Trace Scores (boolean)			
Input Expected Output baseline-gpt4o - 2026-08-07T05:47:06.202042Z Baseline			
question: "what's 23*7?" answer: "161" OUTPUT "23 * 7 equals 161." SCORES correctness 1.00 METADATA Item ID 8c64aa63-eec8-492b-9549-7d8c8b581bbe Observation 0b85516cf67ac449 Execution Trace 6d09055d5e462fa81a8a7273dc944f7e Total Cost \$0.001645 Latency 6.96s			
question: "what's the weather in chennai and what's 100/7?" answer: "Chennai is 22°C and humid; 100/7 is approximately 14.29." OUTPUT "Error: weather service timeout for city='Chennai'" SCORES correctness - METADATA Item ID a0e8a271-4021-4c11-a300-f765329a1a3c Observation c21c8a4bcb3bbf6 Execution Trace 2daea7e1edec112cea63b4586cfea900 Total Cost \$0.001153 Latency 5.97s			
Rows per page 20 Page 1 of 1			

Each row shows the question, the expected answer, what the agent actually said, the correctness score, cost, latency, and a link straight to the execution trace.

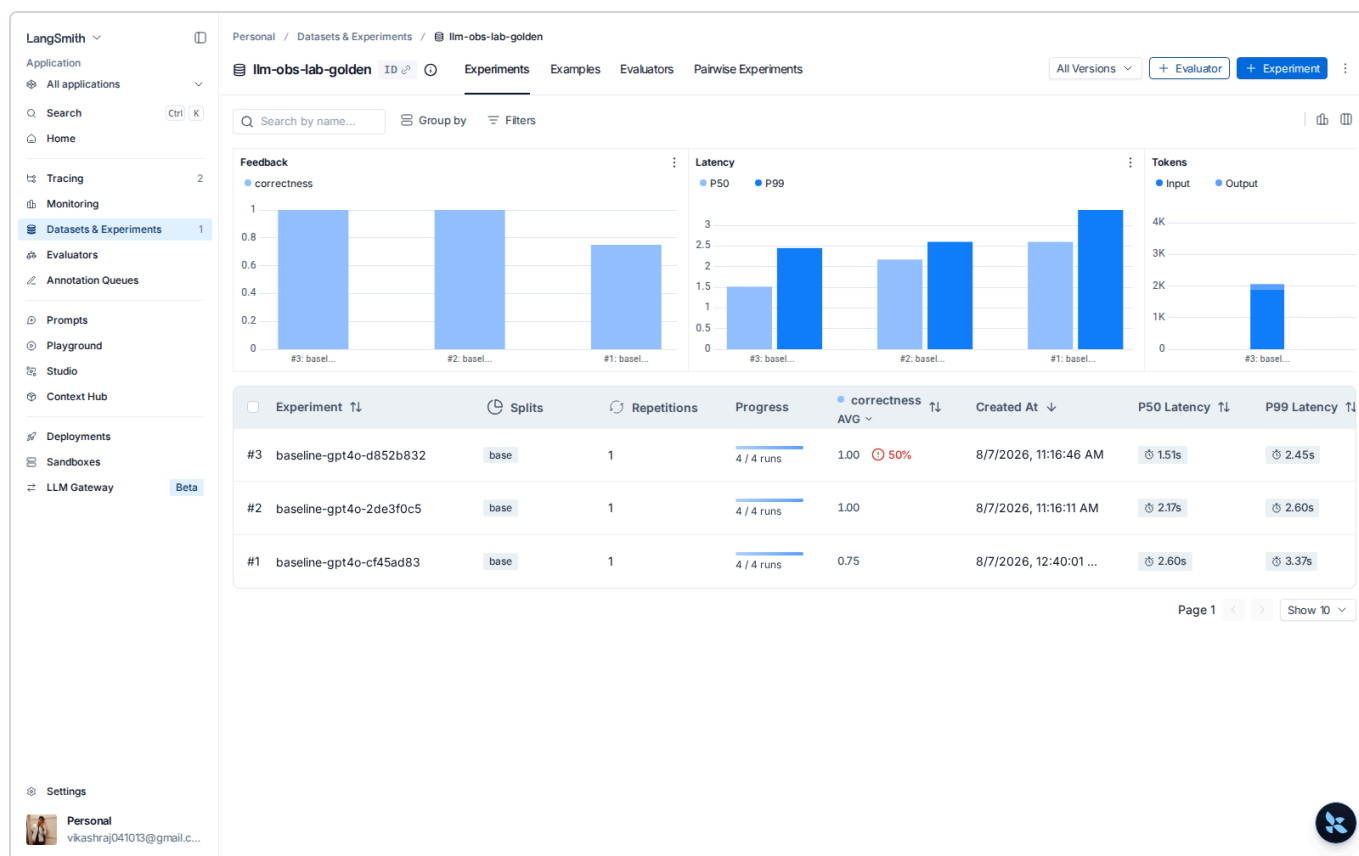
Note the second row. The agent output is "Error: weather service timeout for city='Chennai'" and the score is blank. **The failed case is still in the results.** It is not hidden.

And across runs:



Both runs show Item Count 4 . The Error Count column (10 and 5) is how you spot a flaky agent at a glance.

LangSmith gives you the same thing with more charting on top:



Three experiments, each 4 / 4 runs, with average correctness of 1.00, 1.00 and 0.75, plus P50 and P99 latency per experiment. The red 50% badge on baseline-gpt4o-d852b832 is the forced-failure run. LangSmith surfaces error rate as a first class column next to the score, which is the fastest way to tell "my agent is wrong" apart from "my agent is broken".

The charts along the top (Feedback, Latency, Tokens) update as you add experiments, so regressions between runs are visible without opening anything.

The one real advantage Langfuse has here

Langfuse evaluators return an object with a comment field, so the judge explains itself:

```
return Evaluation(name="correctness", value=0.0, comment="Did not give the temperature.")
```

LangSmith evaluators return a bare bool. A 0.0 tells you it failed but not why. You can work around it, but Langfuse gives it to you by default.

Real numbers from the evaluation

Pulled from the LangSmith API, not typed by hand.

Clean run, everything succeeded:

Question	Score	Cost
weather in Mumbai, then multiply the temperature by 2	0.0	\$0.00278
what do my docs say about retries?	1.0	\$0.00290
what's 23*7?	1.0	\$0.00168
what's the weather in Chennai and what's 100/7?	1.0	\$0.00230

3 out of 4. The Mumbai case genuinely failed the judge. That is the eval doing its job.

Same dataset with the weather tool forced to fail every time:

Question	Result	Score
weather in Mumbai, then multiply the temperature by 2	ERROR	none
what do my docs say about retries?	ok	1.0
what's 23*7?	ok	1.0
what's the weather in Chennai and what's 100/7?	ERROR	none

Both tools behave the same way here, and this corrects a common assumption. Neither aborts the experiment. Both keep all 4 rows. Both leave the score empty on the failed ones.

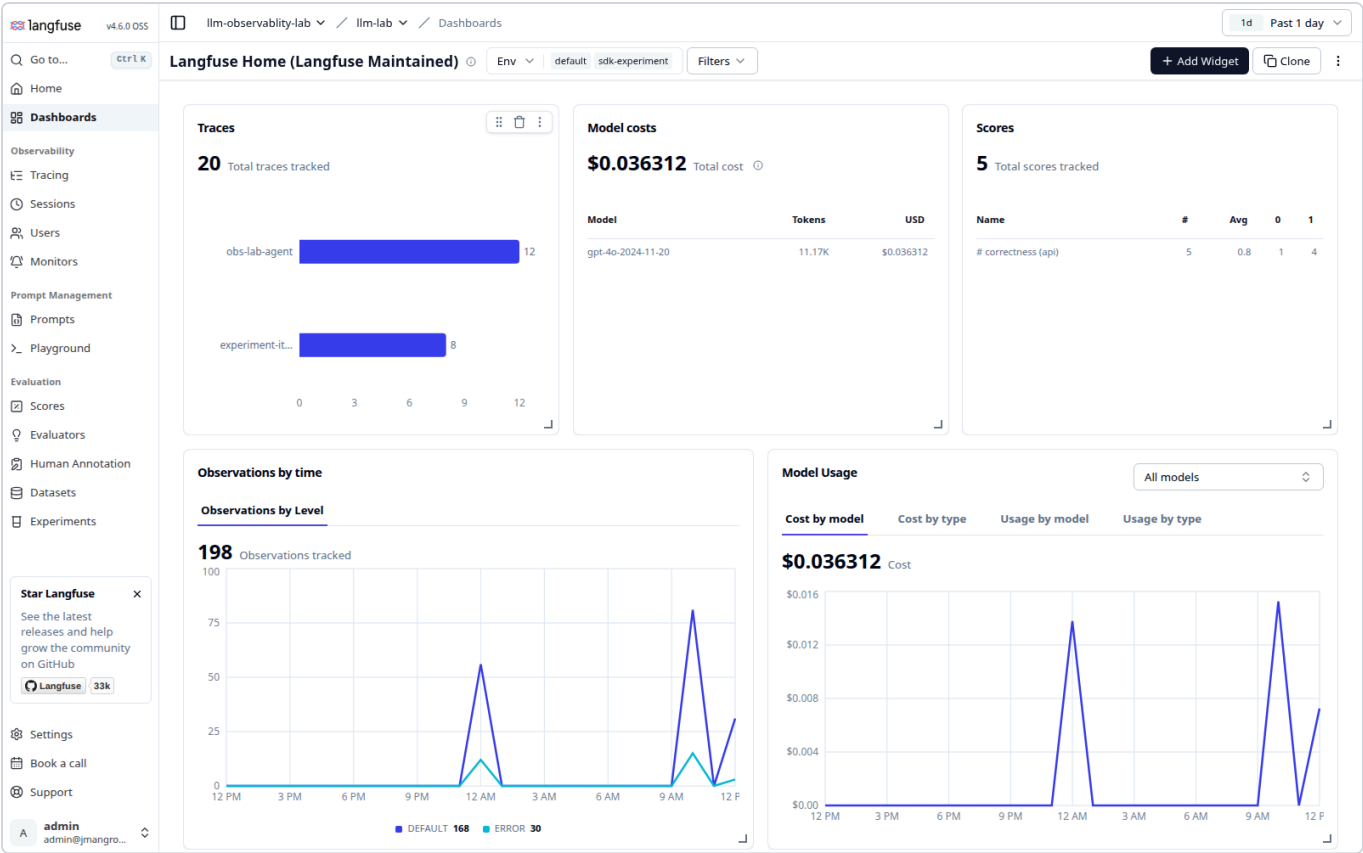
The only difference is in Python. Langfuse drops failed items from the `result.item_results` list, so the script prints `experiment finished: 2 items` even though the UI correctly shows 4. If you log that number and never look at the UI, you will think your dataset shrank. LangSmith keeps all 4 in the returned results, but then runs your evaluator on the failed row too, which throws a second error unless you guard it:

```
answer=outputs.get("answer", "") # not outputs["answer"]
```

Dashboards and feature gaps

Where the two stop being the same kind of product.

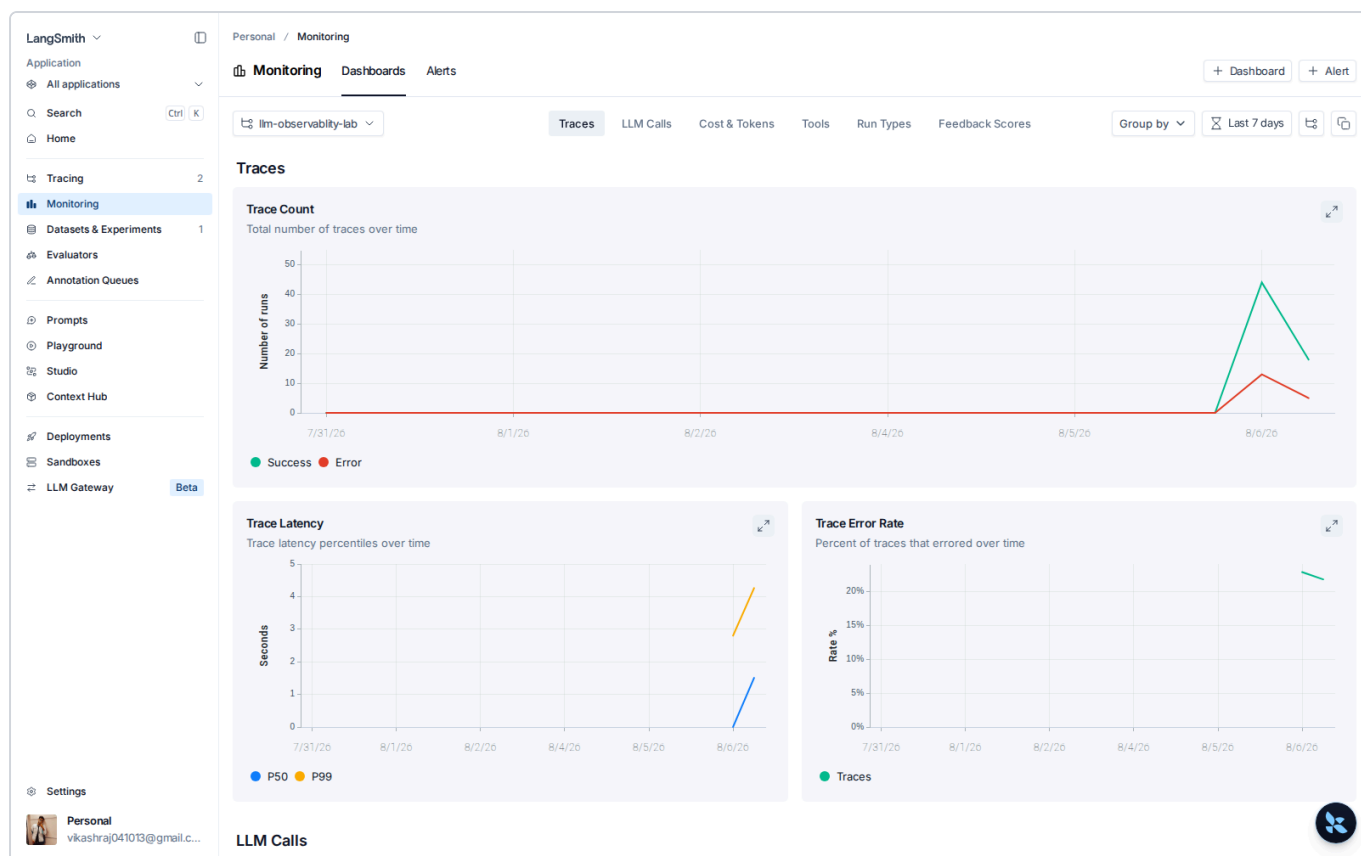
Dashboards



20 traces, \$0.036312 total, 11.17K tokens, 198 observations split DEFAULT 168 / ERROR 30 , and average score 0.8 .

That error split is the single most useful number on the page. 30 out of 198 observations errored, which is the flaky weather tool showing up as a metric instead of a surprise.

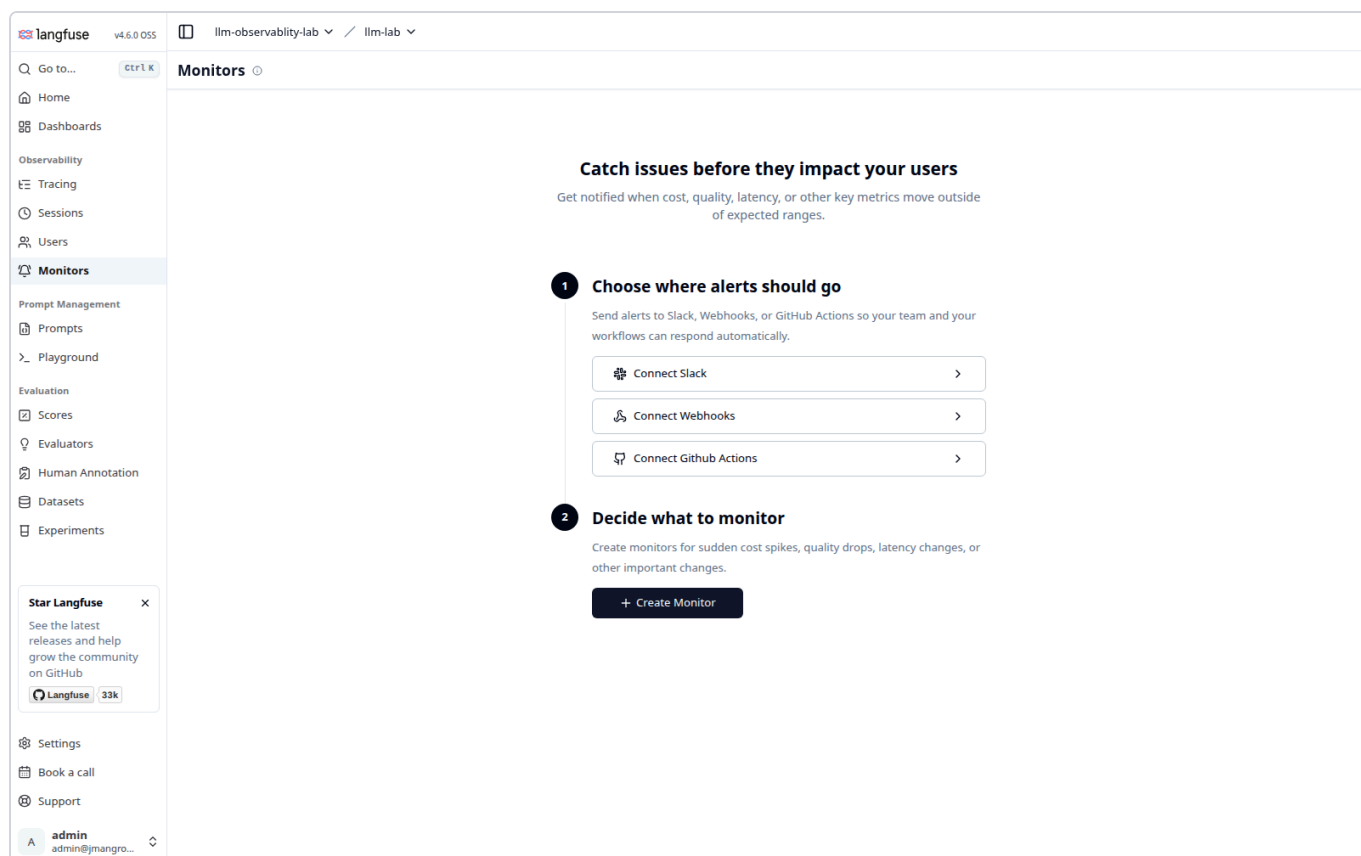
LangSmith calls this Monitoring, and it goes deeper:



Tabs for Traces, LLM Calls, Cost & Tokens, Tools, Run Types and Feedback Scores. Trace Count is split into Success and Error lines, and there is a dedicated Trace Error Rate chart.

Note the second tab at the top: **Alerts**. LangSmith can notify you when error rate or latency crosses a threshold.

Langfuse has Monitors for the same idea:



What each tool has that the other does not

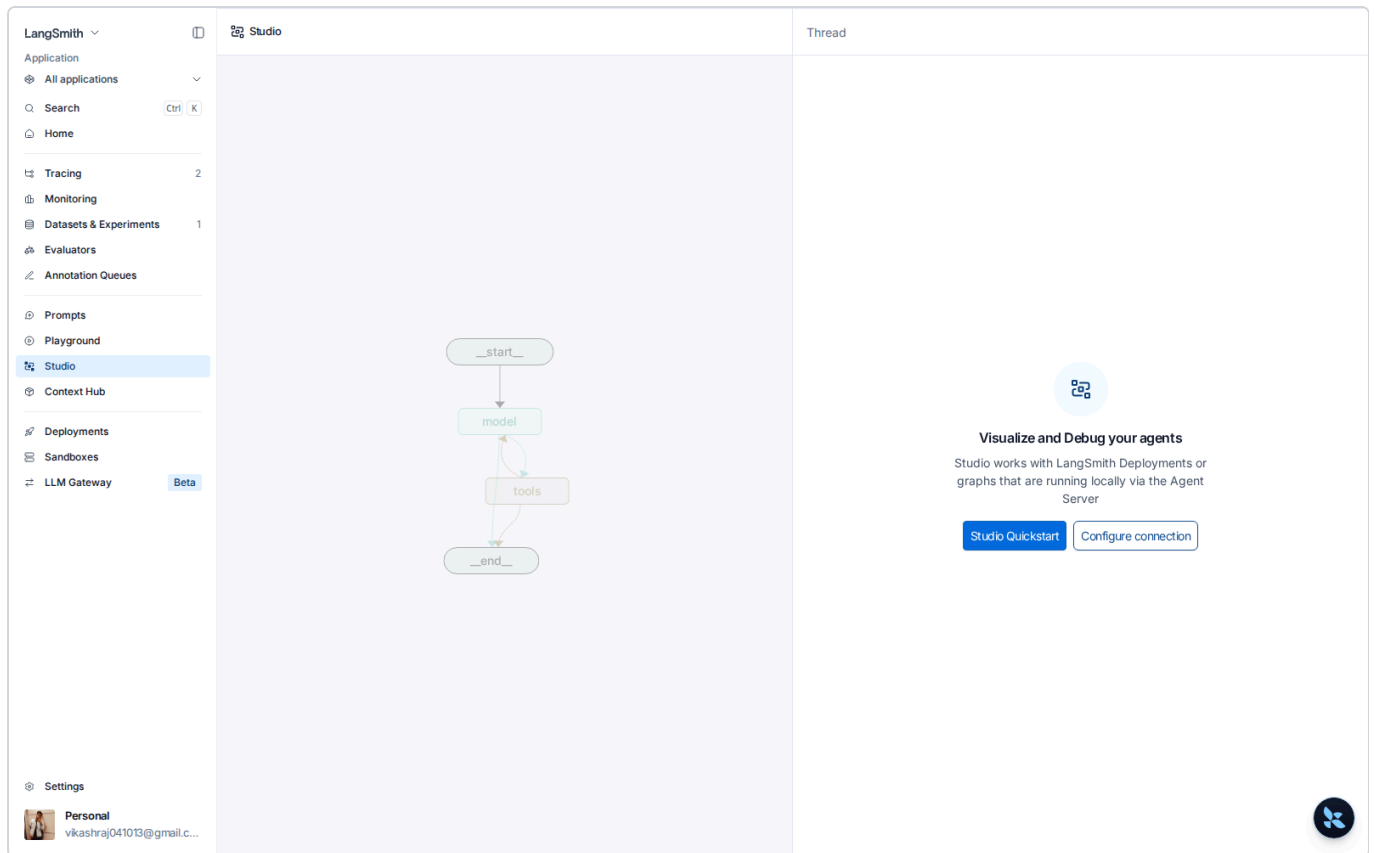
Up to here the two have been close. This is where they stop being the same kind of product.

LangSmith is turning into a platform for building and running agents. Langfuse is an observability and evaluation tool. That is the honest summary, and it cuts both ways.

Side by side, straight from the two menus:

Feature	LangSmith	Langfuse
Tracing	yes	yes
Dashboards and charts	yes (Monitoring)	yes
Alerts on thresholds	yes	yes (Monitors)
Datasets and experiments	yes	yes
LLM-as-judge evaluators	yes	yes
Human annotation	yes (Annotation Queues)	yes
Prompt management	yes	yes
Prompt playground	yes	yes
Grouping runs	Threads	Sessions
Per-user cost tracking	no dedicated page	yes
Self-hosting	no	yes
Visual agent debugger	Studio	no
Agent hosting	Deployments	no
Code sandboxes	Sandboxes	no
LLM proxy	LLM Gateway	no
Context Hub	yes	no

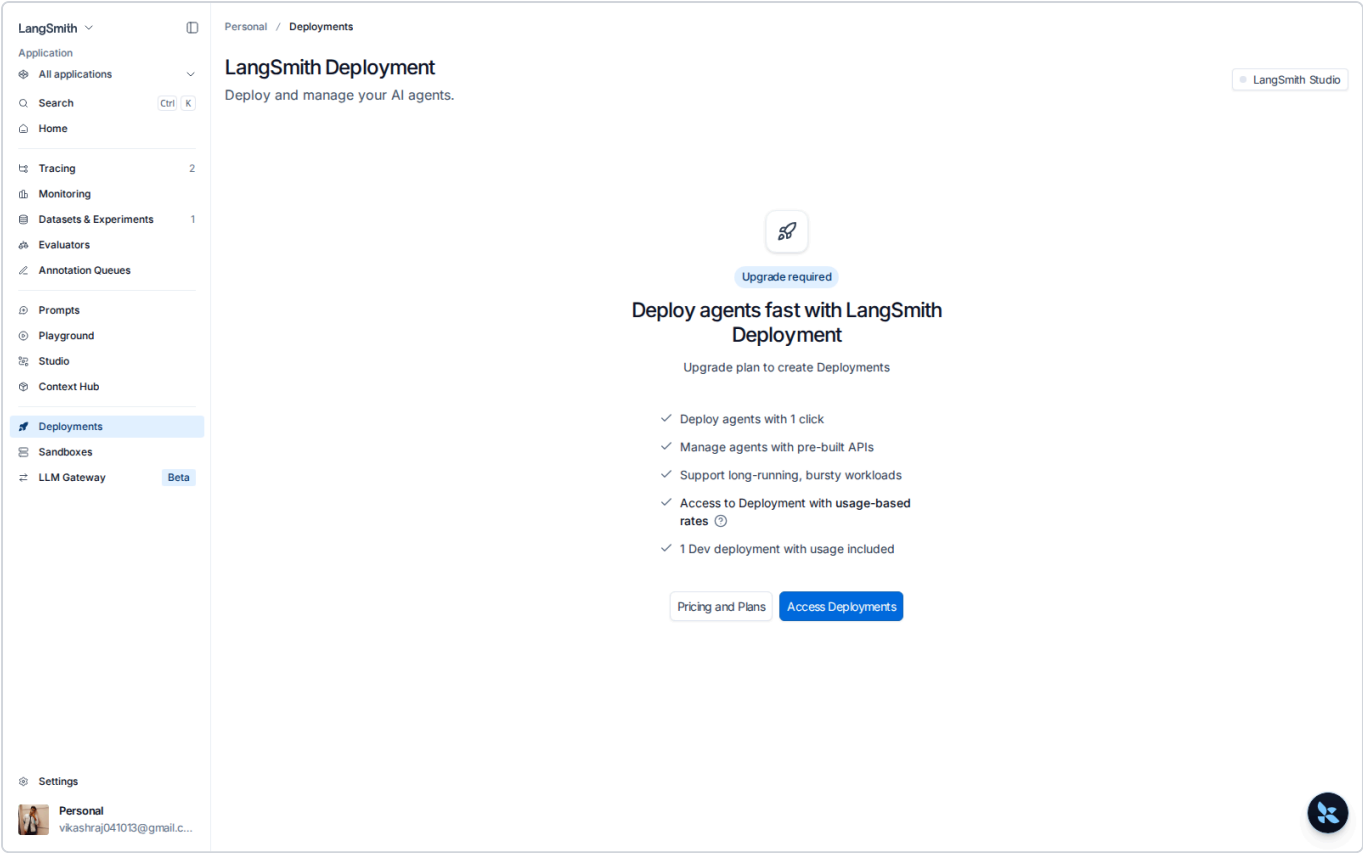
Studio, the one people actually mean



Studio draws your graph (`__start__` → `model` → `tools` → `__end__`) and lets you run it step by step, inspect state at each node, and edit that state before continuing. It connects to either a LangSmith Deployment or a graph running locally through the Agent Server.

This is the biggest single gap. Langfuse can show you a graph of a run that **already happened**. Studio lets you drive the graph **while it runs**, stop it mid-loop and change the state. Those are different jobs. If you build in LangGraph, Studio is a genuine reason to keep LangSmith even if your traces live elsewhere.

Deployments



One-click agent hosting with pre-built APIs, built for long-running and bursty workloads. Note the **Upgrade required** badge: this is a paid tier, not something you get on the free plan.

Sandboxes

LangSmith

Application

All applications

Search

Home

Tracing2

Monitoring

Datasets & Experiments1

Evaluators

Annotation Queues

Prompts

Playground

Studio

Context Hub

Deployments

Sandboxes

LLM GatewayBeta

Settings

Personal

vikashra041013@gmail.c...

Personal / Sandboxes

Sandboxes

Sandboxes are available on plans with sandbox access. Upgrade to unlock sandbox environments for this workspace.

Upgrade

Isolated environments for agents that execute code. Also paid.

LLM Gateway (Beta)

The screenshot displays the LangSmith LLM Gateway interface. On the left is a sidebar with navigation options: Application, All applications, Search, Home, Tracing, Monitoring, Datasets & Experiments, Evaluators, Annotation Queues, Prompts, Playground, Studio, Context Hub, Deployments, Sandboxes, and LLM Gateway (Beta). The main area shows the 'LangSmith LLM Gateway' page with a 'Gateway Credits' section indicating a balance of \$0.00. Below this, there's a 'Bring Your Own Key' section. At the bottom, there are buttons for 'Manage cost controls', 'Configure model fallbacks', and 'Monitor spend'. A code editor in the center shows a TypeScript snippet for using the OpenAI client with the gateway URL.

```
import OpenAI from "openai";

const client = new OpenAI({
  apiKey: process.env.LANGSMITH_API_KEY,
  baseURL: "https://gateway.smith.langchain.com/v1",
});

const response = await client.responses.create({
  model: "moonshotai/Kimi-K3",
  input: "Hello!",
});
```

A proxy in front of model providers. You authenticate to LangSmith instead of holding provider keys, and it handles cost controls, model fallbacks and spend monitoring. Point your OpenAI client at `https://gateway.smith.langchain.com/v1` and it works unchanged.

For this lab that is directly relevant. The agent currently talks to Azure AI Foundry using a key in `.env`. The Gateway is the alternative to managing that key yourself.

What Langfuse has that LangSmith does not

Self-hosting is the headline and the reason this lab runs Langfuse at all. Your traces, prompts and eval results never leave your network. For anything under a data residency rule, that single fact outweighs the entire feature list above.

Per-user cost tracking gets its own page:

langfuse v4.6.0 OSS

llm-observability-lab

llm-lab

1d Past 1 day

Go to... ctrl K

Home

Dashboards

Observability

Tracing

Sessions

Users

Monitors

Prompt Management

Prompts

Playground

Evaluation

Scores

Evaluators

Human Annotation

Datasets

Experiments

Star Langfuse

See the latest releases and help grow the community on GitHub

Langfuse 33k

Settings

Book a call

Support

A admin admin@jmagro...

Users

Search (User ID)

Env default sdk-experiment

Filters

User ID	Environment	First Event	Last Event	Total Events	Total Tokens	Total Cost
vikash	default	8/7/2026, 1:08:53 AM	8/7/2026, 12:43:13 PM	102	6.36K	\$0.02

Rows per page 50

Page 1 of 1

<<

<

>

>>

User `vikash` , 102 events, 6.36K tokens, \$0.02 spent. LangSmith stores `user_id` as ordinary metadata you can filter on. Langfuse aggregates it into a real screen. If you need to know what a specific customer costs you, Langfuse answers it in one click.

Evaluators run against live production traces, configured in the UI:

langfuse

v4.6.0 OSS

llm-observability-lab / llm-lab

Go to...

ctrl K

Home

Dashboards

Observability

Tracing

Sessions

Users

Monitors

Prompt Management

Prompts

Playground

Evaluation

Scores

Evaluators

Human Annotation

Datasets

Experiments

Star Langfuse

See the latest releases and help grow the community on GitHub

Langfuse 33k

Settings

Book a call

Support

admin admin@jmgro...

Evaluators

Get Started with LLM-as-a-Judge Evaluations

Create evaluation templates and evaluators to automatically score your traces with LLM-as-a-judge. Set up custom evaluation criteria and let AI help you measure the quality of your outputs.

Create Evaluator Learn More

Create eval template

Context relevance

name context-relevance

prompt

Example:

George has online service anyone ever used?

Customer: George has none, "but your service, no have great service?" do there any other to this statement or do it a bunch of badger?

VEL: no, service isn't answer your about, about if you have less than perfect video. A list of service isn't use a blind person about video. If your video problem isn't related to video it, you'll have won't charge no actor? how many service you act.

Score: 4.5

Reasoning: The first sentence is introducing the topic of the query but not relevant to answer it. The following statement clearly answers the question and thus, is relevant. The rest of the sentence are strengthening the conclusion and thus, also relevant.

Output:

George: (sarcastic)

(sarcastic: Relevant)

Think step by step.

Model

Provider openai

Model name gpt-4o

Select a model which supports function calling

Temperature 0

Maximum batch 250

Top P 1

API key

The LLM API key is used for each evaluator and all trace runs.

Variables

query context

Score

provide a score between 0 and 5

You use function calls to extract data from the LLM. Specify what the LLM should return for the score.

Automate evaluations

Use LLM-as-a-judge to automatically evaluate your traces without manual review

Measure quality

Create custom evaluation criteria to measure the quality of your LLM outputs

Scale efficiently

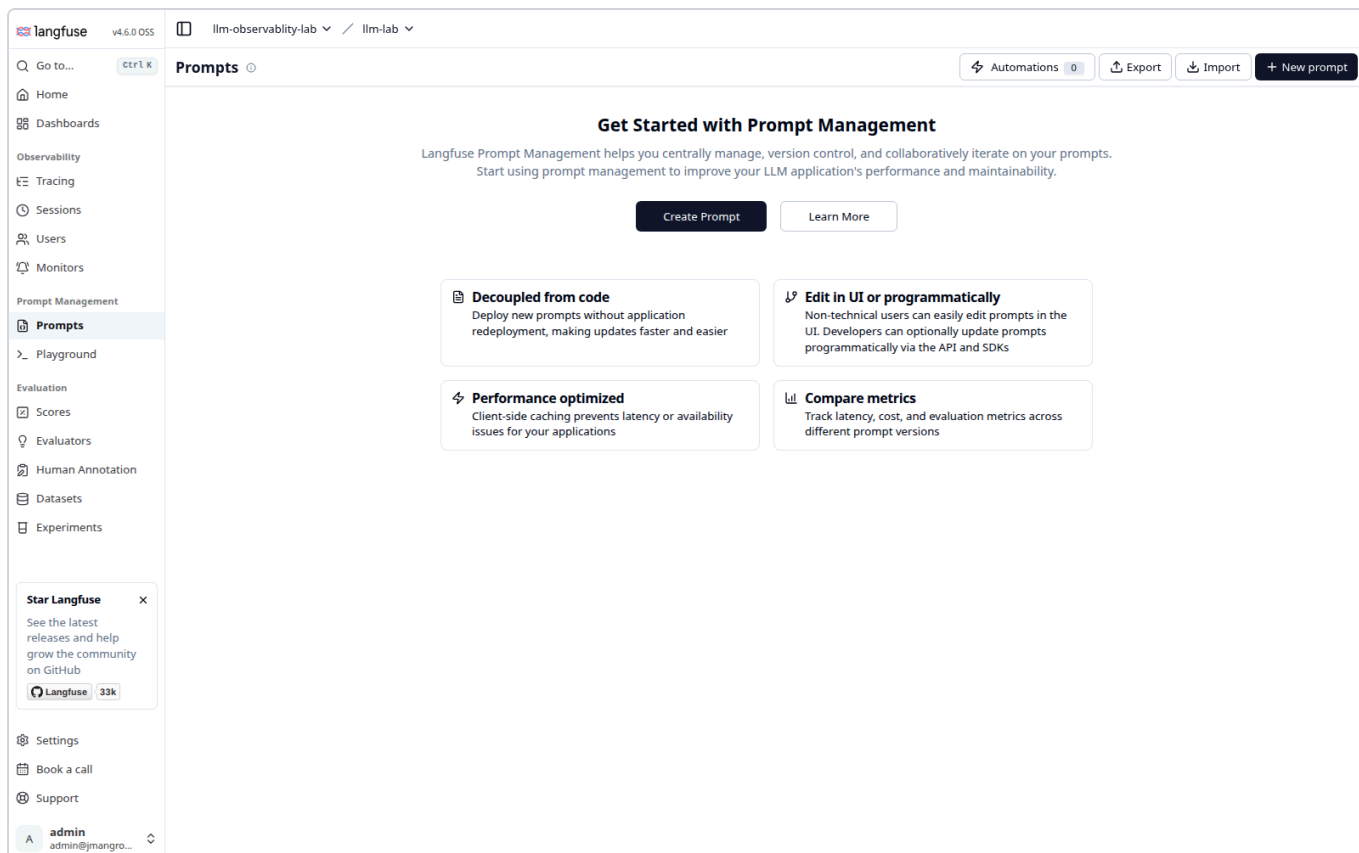
Evaluate thousands of traces automatically with customizable sampling rates

Track performance

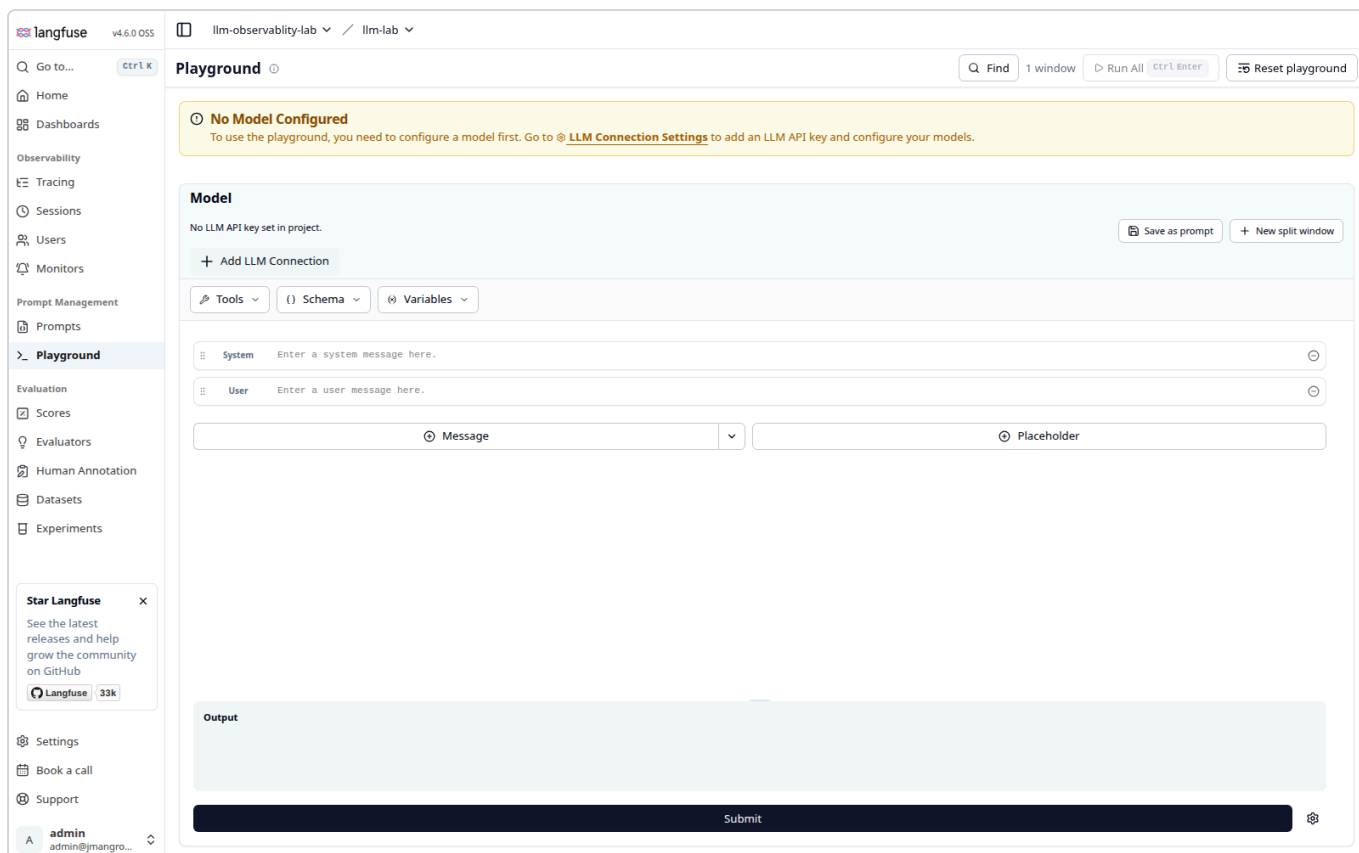
Monitor evaluation metrics over time to identify trends and improvements

You build the judge template in a form (prompt, model, temperature, sampling rate) and it scores real traffic automatically. No code, no dataset required. LangSmith has evaluators too, but the Langfuse version is aimed squarely at continuous scoring of production traces rather than at offline dataset runs.

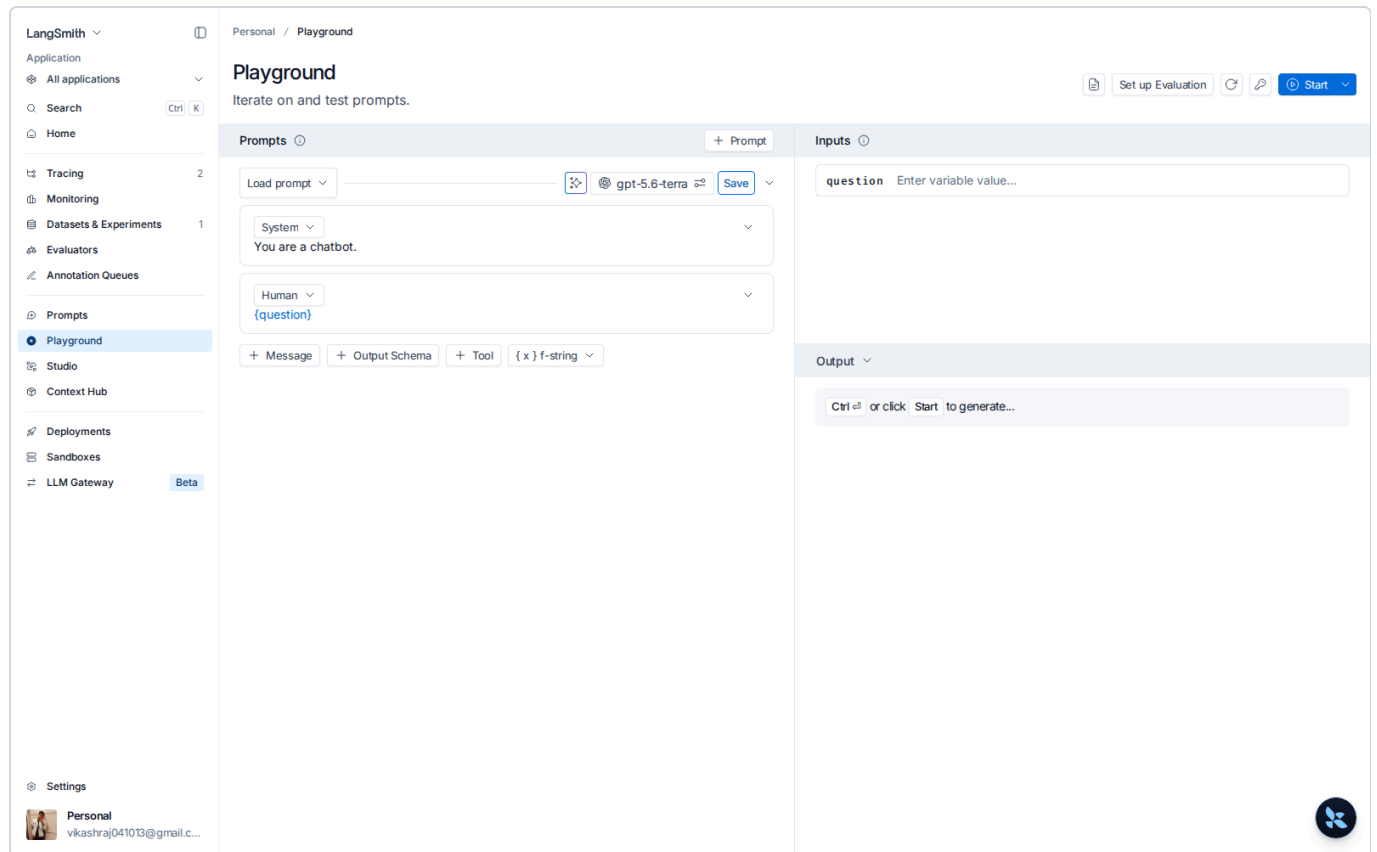
Prompt management and Playground exist on both sides, so neither wins here:



Langfuse Playground:



LangSmith Playground:



How to choose

- Building on LangGraph and you want to debug agents visually, host them, or proxy your models? **LangSmith**, and it is not close. Studio alone decides it.
- Data cannot leave your network, or you need per-user cost attribution, or you want judges scoring live traffic without writing code? **Langfuse**.
- Just want to see what your agent did and score it against a dataset? Either. They are close enough that the deciding factor is hosting, not features.

Pricing and running costs

Published prices as of August 2026, plus what the Langfuse stack actually consumes on this machine.

Pricing

Prices as published in August 2026. Check them before you commit to anything.

LangSmith

Plan	Price	Included traces	Seats	Self-hosting
Developer	\$0 then pay as you go	5k base traces / month	1 only	no
Plus	\$39 / seat / month then pay as you go	10k base traces / month	unlimited	no
Enterprise	custom	custom	custom	yes

Past the included allowance you pay per trace, and retention is what drives the price:

Trace type	Price	Retention
Base	\$0.50 per 1,000 traces (0.05 cents each)	14 days
Extended	\$5.00 per 1,000 traces (0.50 cents each)	400 days

Extended retention is **10x** the base price. Upgrading a base trace afterwards costs 0.45 cents. Traces with feedback attached (evaluator scores, human labels) get upgraded, which is exactly what your eval runs produce. Worth knowing before you run a large experiment.

Two things gated behind Enterprise: **self-hosting** and **custom SSO**. Deployments and Sandboxes are paid on top of the plan. On the Developer plan you get 5 LCU and 1 LSU of sandbox usage a month, capped at 10 sandboxes.

Langfuse

Self-hosted:

Tier	Price	What you get
Open Source	Free, MIT licence	Everything in this document. Unlimited projects, users, traces.
Self-hosted Enterprise	custom	RBAC, audit logs, data masking, retention policies, SOC 2 / ISO 27001 docs, SLA

Cloud:

Plan	Price	Included units	Extra units	Retention
Hobby	\$0	50k / month	none	30 days
Core	\$29 / month	100k / month	\$8 per 100k	90 days
Pro	\$199 / month	100k / month	\$8 per 100k	3 years
Enterprise	\$2,499 / month	100k / month	\$8 per 100k	3 years

The trap: units are not traces

LangSmith bills per **trace**. Langfuse bills per **unit**, which is roughly one observation (one span).

This lab's own dashboard settles it: **20 traces produced 198 observations**, so this agent averages **9.9 units per trace**.

That changes the free tiers completely:

	Free allowance	In traces, for this agent
LangSmith Developer	5k traces	5,000 traces / month
Langfuse Cloud Hobby	50k units	~5,050 traces / month
Langfuse Cloud Core (\$29)	100k units	~10,100 traces / month

The free tiers land within one percent of each other. A per-unit price that looks ten times cheaper is not, once your agent has a few tool calls in the loop. A chattier agent makes this worse.

What Langfuse actually costs to run

Measured on this machine (14 cores, 15 GiB RAM) with the stack idle after the lab runs.

Memory and CPU, per container

Container	Memory	CPU
langfuse-web	888.3 MiB	0.24%
clickhouse	669.4 MiB	4.61%
langfuse-worker	533.0 MiB	0.26%
minio	78.5 MiB	3.64%
postgres	47.1 MiB	0.00%
redis	6.4 MiB	2.06%
Total	2.16 GiB	~11% of one core

Disk

What	Size
Docker images (6)	~6.0 GB
Git clone	261 MB
ClickHouse data + logs	360 MB
Postgres data	46 MB
MinIO + Redis data	< 1 MB
Total after 20 traces	~6.7 GB

Note the shape of that. Almost all of it is **images, not data**. `langfuse-worker` alone is 2.07 GB and `langfuse-web` is 1.75 GB. You pay 6 GB of disk before recording a single trace.

Measured versus recommended

Langfuse officially recommends **4 cores, 16 GiB RAM, and 100 GiB storage** (a `t3.xlarge` class VM).

The gap is large: 2.16 GiB measured against 16 GiB recommended. Both numbers are honest. The recommendation covers production ingest, where ClickHouse is absorbing continuous writes rather than sitting idle. For a lab or a small team, the real figure is closer to what is measured here.

The practical read on a 16 GiB laptop:

- Langfuse takes about **14%** of total RAM at idle
- That is fine alongside an IDE and a browser, but it is not free
- **ClickHouse is the reason this is six containers and not one binary**. It is the largest memory user, the largest disk user, and the busiest at idle. It is also what makes trace queries fast at volume.

The docs are explicit that docker compose is for trying Langfuse out, not for running it. It has **no high availability, no horizontal scaling, and no backup**. Production means Kubernetes.

Operational friction, measured in this lab

	LangSmith	Langfuse
Setup	Nothing to run	6 containers, ~6.7 GB
RAM	0	2.16 GiB idle, 16 GiB recommended
Ports claimed	none	3000, 3030, 5432, 8123, 9000, 9090, 9091, 6379
Your data	leaves your network	stays local
Ongoing work	none	port clashes, container health, backups are your problem

Port `5432` is the one that actually bit. Any other local Postgres stops the whole stack from starting, and Compose aborts the entire `up` when one container fails to bind.

When does self-hosting actually save money

The licence is free. The machine is not.

A 4 core / 16 GiB VM plus 100 GiB of storage is the floor, and at typical cloud on-demand rates that lands somewhere around **\$100 to \$150 a month**. That figure is an estimate from published instance pricing, not something measured here.

Compare that against Langfuse Cloud Core at **\$29 a month** for 100k units, plus **\$8 per additional 100k**:

```
self-hosted VM  ~= $130 / month, flat
Langfuse Cloud  =  $29 + $8 per extra 100k units
break-even      ~=  1.3 million units / month
                ~=  130,000 traces / month for this agent
```

Below roughly 130k traces a month, self-hosting costs more, not less. You are paying for a VM that is mostly idle.

So the reason to self-host is almost never the bill. It is that the data cannot leave your network. If that constraint is real, the cost question does not arise. If it is not real, cloud is cheaper until you are at serious volume.

One more thing the price lists do not show: **someone has to run it**. Upgrades, backups, disk filling with ClickHouse data, the 3am page when the worker dies. That is not on any pricing page and it usually costs more than the VM.
